

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA



DEPARTAMENTO DE
ELECTRONICA

Tarea: Emulando Redes con GNS3

Redes de Computadores II / Seminario de Redes de Computadores

Fecha: 18 de octubre de 2026
Integrantes: Vicente Andrade; Ernesto Gamero; Matías Pérez
Asignatura: ELO323 / IPD438
Profesor: Agustín González V.
Grupo: 3

Índice

1 Desarrollo **3**
1.1 Familiarización con redes LAN . . . 3
1.1.1 Crear LANs independientes (1.a) . . . 3
1.1.2 Añadir un router (1.b) . . . 3
1.1.3 Asignación de direcciones IP por DHCP (1.c) . . . 4
1.1.4 Pruebas de conectividad (1.d) . . . 6
1.2 Simulación de enlaces ideales y reales . . . 6
1.2.1 Pruebas de conectividad (2.a) . . . 7
1.2.2 Intervalos de confianza (2.b) . . . 8
1.2.3 Mediciones de throughput (2.c) . . . 9

Índice de figuras

1 Topología con dos LANs sin conexión entre sí (1.a). . . . 3
2 Configuración de PC2 y pings hacia los otros VPCS (1.a). . . . 3
3 Comandos ejecutados en la consola de R1 (1.b). . . . 4
4 Asignación de direcciones por DHCP (1.c). . . . 5
5 Topología completa con las direcciones obtenidas por DHCP (1.c). . . . 5
6 Pruebas de conectividad desde PC2 (1.d). . . . 6
7 Topología de la pregunta 2 en GNS3, con las direcciones y la configuración del enlace NETem. 7
8 Pruebas desde PC2 en la red con el enlace NETem. . . . 8
9 Mediciones de throughput con TCP (2.c). . . . 10
10 Mediciones de throughput con UDP desde PC3 (2.c). . . . 11

Índice de tablas

1 Direccionamiento de la red en la pregunta 1. . . . 4
2 Pruebas TCP de 60 s desde PC3 hacia PC1 (2.c). . . . 9

1. Desarrollo

Las topologías se armaron en GNS3 2.2.61 corriendo directamente sobre Linux, sin la GNS3 VM. Los terminales son VPCS, los switches son nodos Ethernet switch de GNS3, que conmutan en capa 2 sin cargar ningún sistema operativo, y el router es un Cisco 3725 emulado con dynamips.

1.1. Familiarización con redes LAN

1.1.1. Crear LANs independientes (1.a)

PC1 y PC2 quedaron conectados a Switch1, y PC3 y PC4 a Switch2. Entre los dos switches no hay enlace, de modo que cada uno es un dominio de broadcast aislado (Figura 1). Las direcciones se asignaron a mano en la consola de cada VPCS, por ejemplo con `ip 192.168.0.102/24` en PC2, sin indicar gateway, y se revisaron con `show ip`.

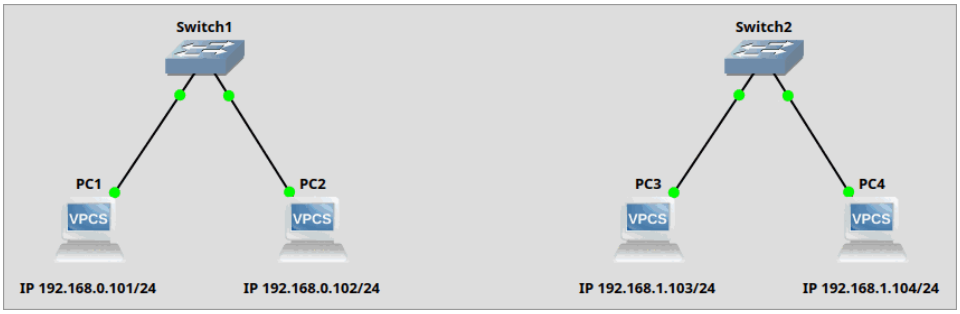


Figura 1: Topología con dos LANs sin conexión entre sí (1.a).

El ping de PC2 a PC1 (192.168.0.101) recibió las cinco respuestas con TTL 64 y tiempos entre 0,17 y 0,27 ms (Figura 2). Hacia PC3 y PC4 el resultado fue `No gateway found`, y VPCS no llegó a enviar ningún paquete.

Antes de transmitir, PC2 aplica su máscara /24 al destino. Como 192.168.0.101 pertenece a su misma red, le basta resolver la MAC de PC1 por ARP y Switch1 entrega la trama directamente. En cambio 192.168.1.103 y 192.168.1.104 están fuera de 192.168.0.0/24, así que el paquete tendría que ir a un gateway, pero PC2 no tiene uno (`show ip` indica `GATEWAY: 0.0.0.0`). Aunque se hubiera configurado, nada en esta topología podría reenviar tráfico entre las dos redes, porque los switches no están unidos y ninguno trabaja en capa 3.

```
PC2> ip 192.168.0.102/24
Checking for duplicate address...
PC2 : 192.168.0.102 255.255.255.0

PC2> show ip

NAME        : PC2[1]
IP/MASK     : 192.168.0.102/24
GATEWAY     : 0.0.0.0
DNS         :
MAC         : 00:50:79:66:68:01
LPORT       : 10010
RHOST:PORT  : 127.0.0.1:10011
MTU         : 1500

PC2> ping 192.168.0.101

84 bytes from 192.168.0.101 icmp_seq=1 ttl=64 time=0.197 ms
84 bytes from 192.168.0.101 icmp_seq=2 ttl=64 time=0.266 ms
84 bytes from 192.168.0.101 icmp_seq=3 ttl=64 time=0.170 ms
84 bytes from 192.168.0.101 icmp_seq=4 ttl=64 time=0.227 ms
84 bytes from 192.168.0.101 icmp_seq=5 ttl=64 time=0.172 ms

PC2> ping 192.168.1.103

No gateway found

PC2> ping 192.168.1.104

No gateway found

PC2>
```

Figura 2: Configuración de PC2 y pings hacia los otros VPCS (1.a).

1.1.2. Añadir un router (1.b)

Con los VPCS apagados se agregó el router R1. Su template se importó desde el appliance `cisco-3725.gns3a` con la imagen `c3725-adventerprisek9-mz.124-15.T14` (IOS 12.4(15)T14), cuyo MD5 coincide con el que declara el appliance. Se conservó su valor de idle-PC (0x60c09aa0), sin el cual dynamips mantiene ocupado un núcleo completo del computador. La RAM se subió de 128 a 256 MB porque el material de ayuda lo pide para la pregunta 4. La interfaz FastEthernet0/0 (f0/0) se conectó al puerto e0 de Switch1 y la FastEthernet0/1 (f0/1) al e0 de Switch2.

En la consola (Figura 3), cada interfaz recibió la primera dirección asignable de su red y un `no shutdown`, necesario porque en IOS las interfaces del router parten deshabilitadas (al arrancar aparecen como *adminis-*

tratively down). Después de ese comando, los mensajes `%LINK-3-UPDOWN` y `%LINEPROTO-5-UPDOWN` confirman que cada enlace quedó arriba.

Para el servicio DHCP se definió un pool por subred, LAN0 y LAN1. En cada uno, `network` fija la subred y la máscara que se entregan, y es lo que IOS usa para elegir el pool: la compara con la dirección de la interfaz por la que llega la solicitud. `default-router` define el gateway que recibirá el cliente y `dns-server` el servidor de nombres, que aquí es 8.8.8.8. El enunciado pide configurarlo aunque la red no tiene salida a Internet, así que ese valor no interviene en las pruebas. La dirección .1 de cada red, que ya ocupa el router, se excluyó del pool con `ip dhcp excluded-address`. La configuración se guardó en NVRAM con `copy running-config startup-config`, que respondió `[OK]`.

```
R1#enable
R1#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#interface FastEthernet0/0
R1(config-if)#ip address 192.168.0.1 255.255.255.0
R1(config-if)#no shutdown
R1(config-if)#exit
R1(config)#
*Mar  1 00:00:50.743: %LINK-3-UPDOWN: Interface FastEthernet0/0, changed state to up
*Mar  1 00:00:51.755: %LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/0, changed state
 to up
R1(config)#interface FastEthernet0/1
R1(config-if)#ip address 192.168.1.1 255.255.255.0
R1(config-if)#no shutdown
R1(config-if)#exit
R1(config)#
*Mar  1 00:00:56.731: %LINK-3-UPDOWN: Interface FastEthernet0/1, changed state to up
*Mar  1 00:00:57.731: %LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/1, changed state
 to up
R1(config)#end
R1#
*Mar  1 00:00:58.747: %SYS-5-CONFIG_I: Configured from console by console
R1#
```

(a) Direcciones de las interfaces FastEthernet.

```
R1#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#ip dhcp excluded-address 192.168.0.1
R1(config)#ip dhcp excluded-address 192.168.1.1
R1(config)#ip dhcp pool LAN0
R1(dhcp-config)#network 192.168.0.0 255.255.255.0
R1(dhcp-config)#default-router 192.168.0.1
R1(dhcp-config)#dns-server 8.8.8.8
R1(dhcp-config)#exit
R1(config)#ip dhcp pool LAN1
R1(dhcp-config)#network 192.168.1.0 255.255.255.0
R1(dhcp-config)#default-router 192.168.1.1
R1(dhcp-config)#dns-server 8.8.8.8
R1(dhcp-config)#exit
R1(config)#end
R1#
*Mar  1 00:01:19.835: %SYS-5-CONFIG_I: Configured from console by console
R1#copy running-config startup-config
Destination filename [startup-config]?
Building configuration...
[OK]
R1#
```

(b) Pools DHCP y respaldo de la configuración.

Figura 3: Comandos ejecutados en la consola de R1 (1.b).

1.1.3. Asignación de direcciones IP por DHCP (1.c)

Con R1 encendido, cada VPCS pidió su dirección con `ip dhcp`. La línea `DORA IP 192.168.0.2/24 GW 192.168.0.1` resume la negociación completa de PC1 (Figura 4a), porque VPCS imprime una letra por cada mensaje del intercambio (Discover, Offer, Request y Ack). En `show ip` aparece también lo demás que entregó el servidor: DNS 8.8.8.8, la dirección del servidor DHCP y un arriendo de 86400 s, es decir, un día, que es el valor por defecto de IOS.

Las cuatro asignaciones se ven en la tabla de R1 (Figura 4b). El Client-ID de cada entrada empieza con 01, el tipo de hardware Ethernet, y sigue con la MAC del VPCS (de 00:50:79:66:68:00 a 00:50:79:66:68:03), así que cada arriendo se puede asociar a su equipo. IOS fue asignando las direcciones de cada pool en orden ascendente; como se pidieron de PC1 a PC4, PC1 y PC3 quedaron con la .2 y PC2 y PC4 con la .3. Mientras R1 conserve esos arriendos, cada VPCS vuelve a recibir la misma dirección. Los arriendos viven solo en la RAM del router (no se configuró `ip dhcp database`), así que después de reiniciar R1 un orden distinto de solicitudes intercambiaría las direcciones. La Tabla 1 reúne las direcciones de esta etapa junto con las manuales de la pregunta 1.a.

Tabla 1: Direccionamiento de la red en la pregunta 1.

Equipo	Conexión	IP en 1.a (manual)	IP en 1.b y 1.c	Gateway en 1.c
PC1	Switch1 e1	192.168.0.101/24	192.168.0.2/24 (DHCP)	192.168.0.1
PC2	Switch1 e2	192.168.0.102/24	192.168.0.3/24 (DHCP)	192.168.0.1
PC3	Switch2 e1	192.168.1.103/24	192.168.1.2/24 (DHCP)	192.168.1.1
PC4	Switch2 e2	192.168.1.104/24	192.168.1.3/24 (DHCP)	192.168.1.1
R1 f0/0	Switch1 e0	no existía	192.168.0.1/24 (manual)	no aplica
R1 f0/1	Switch2 e0	no existía	192.168.1.1/24 (manual)	no aplica

Cada VPCS guardó su configuración con `save pc_i.vpc`. El archivo contiene el nombre del equipo y la instrucción `dhcp`, sin la dirección, así que al cargarlo el VPCS vuelve a pedir una. La Figura 4c lo muestra en PC1 después de apagarlo y encenderlo: `show ip` vuelve a 0.0.0.0/0 y, tras `load pc_1.vpc`, el equipo recupera

192.168.0.2 porque R1 todavía tenía el arriendo asociado a su MAC. Los cuadros de texto del diagrama se actualizaron con la IP y el gateway de cada equipo (Figura 5).

```
PC1> ip dhcp
DORA IP 192.168.0.2/24 GW 192.168.0.1

PC1> show ip

NAME        : PC1[1]
IP/MASK     : 192.168.0.2/24
GATEWAY     : 192.168.0.1
DNS         : 8.8.8.8
DHCP SERVER : 192.168.0.1
DHCP LEASE  : 86385, 86400/43200/75600
MAC         : 00:50:79:66:68:00
LPORT       : 10008
RHOST:PORT  : 127.0.0.1:10009
MTU         : 1500

PC1> save pc_1.vpc
Saving startup configuration to pc_1.vpc
.  done

PC1>
```

(a) `ip dhcp` y `save` en PC1.

```
R1#show ip dhcp binding
Bindings from all pools not associated with VRF:
IP address          Client-ID/              Lease expiration        Type
                    Hardware address/
                    User name
192.168.0.2         0100.5079.6668.00       Mar 02 2002 12:14 AM    Automatic
192.168.0.3         0100.5079.6668.01       Mar 02 2002 12:15 AM    Automatic
192.168.1.2         0100.5079.6668.02       Mar 02 2002 12:15 AM    Automatic
192.168.1.3         0100.5079.6668.03       Mar 02 2002 12:16 AM    Automatic
R1#
```

(b) Arriendos registrados en R1.

```
PC1> show ip

NAME        : PC1[1]
IP/MASK     : 0.0.0.0/0
GATEWAY     : 0.0.0.0
DNS         :
MAC         : 00:50:79:66:68:00
LPORT       : 10008
RHOST:PORT  : 127.0.0.1:10009
MTU         : 1500

PC1> load pc_1.vpc

Executing the file "pc_1.vpc"


DORA IP 192.168.0.2/24 GW 192.168.0.1

PC1> show ip

NAME        : PC1[1]
IP/MASK     : 192.168.0.2/24
GATEWAY     : 192.168.0.1
DNS         : 8.8.8.8
DHCP SERVER : 192.168.0.1
DHCP LEASE  : 86385, 86400/43200/75600
MAC         : 00:50:79:66:68:00
LPORT       : 10008
RHOST:PORT  : 127.0.0.1:10009
MTU         : 1500

PC1>
```

(c) `load` tras reiniciar PC1.

Figura 4: Asignación de direcciones por DHCP (1.c).

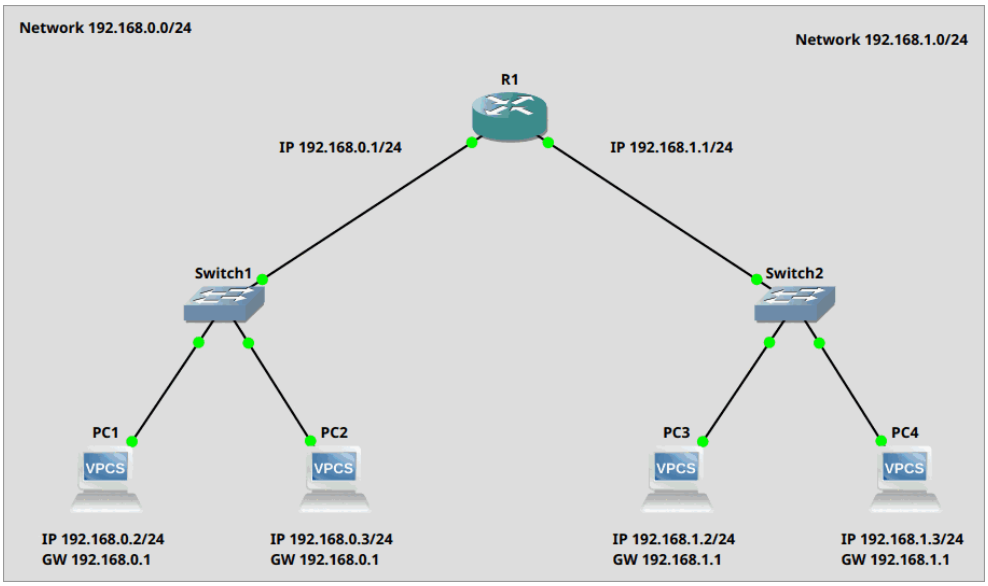


Figura 5: Topología completa con las direcciones obtenidas por DHCP (1.c).

El proyecto se adjunta como `TAREA_ELO323.IPD438_parte1.gns3project`, exportado sin las imágenes base, que según el material de ayuda no es necesario incluir. Como GNS3 solo exporta con los nodos detenidos, antes se apagó R1; al detenerlo, GNS3 extrae la startup-config de su NVRAM y la escribe en `configs/i1_startup-config.cfg`, de modo que lo que no se hubiera guardado con `copy running-config startup-config` no habría quedado en el proyecto. Del archivo se quitó la imagen *ghost* de dynamips, una copia de 256 MB de la RAM del router con el IOS ya cargado, que GNS3 vuelve a generar al iniciar el router; sin ella el proyecto pesa 30 KB. La exportación se comprobó importándola como proyecto nuevo: R1 arrancó con sus interfaces y pools, y los cuatro VPCS traían su `pc_i.vpc`. Para obtener las mismas direcciones de

la Tabla 1 hay que encender R1, esperar a que IOS termine de cargar y ejecutar `load pc_i.vpc` de PC1 a PC4.

1.1.4. Pruebas de conectividad (1.d)

Los tres pings desde PC2 recibieron todas sus respuestas (Figura 6a). VPCS muestra el TTL de la respuesta, que todos los VPCS emiten con 64. La de PC1 llega con ese valor y tarda menos de 0,5 ms porque el tráfico solo cruza Switch1. Las de PC3 y PC4 llegan con TTL 63, ya que R1, el único router del camino, lo descuenta en uno al reenviarlas de f0/1 a f0/0.

```
PC2> ping 192.168.0.2

84 bytes from 192.168.0.2 icmp_seq=1 ttl=64 time=0.450 ms
84 bytes from 192.168.0.2 icmp_seq=2 ttl=64 time=0.158 ms
84 bytes from 192.168.0.2 icmp_seq=3 ttl=64 time=0.485 ms
84 bytes from 192.168.0.2 icmp_seq=4 ttl=64 time=0.238 ms
84 bytes from 192.168.0.2 icmp_seq=5 ttl=64 time=0.211 ms

PC2> ping 192.168.1.2

84 bytes from 192.168.1.2 icmp_seq=1 ttl=63 time=99.912 ms
84 bytes from 192.168.1.2 icmp_seq=2 ttl=63 time=36.620 ms
84 bytes from 192.168.1.2 icmp_seq=3 ttl=63 time=56.017 ms
84 bytes from 192.168.1.2 icmp_seq=4 ttl=63 time=66.630 ms
84 bytes from 192.168.1.2 icmp_seq=5 ttl=63 time=56.519 ms

PC2> ping 192.168.1.3

84 bytes from 192.168.1.3 icmp_seq=1 ttl=63 time=83.642 ms
84 bytes from 192.168.1.3 icmp_seq=2 ttl=63 time=56.084 ms
84 bytes from 192.168.1.3 icmp_seq=3 ttl=63 time=36.776 ms
84 bytes from 192.168.1.3 icmp_seq=4 ttl=63 time=36.608 ms
84 bytes from 192.168.1.3 icmp_seq=5 ttl=63 time=55.974 ms

PC2>
```

(a) `ping` hacia PC1, PC3 y PC4.

```
PC2> trace 192.168.1.2
trace to 192.168.1.2, 8 hops max, press Ctrl+C to stop
 1   192.168.0.1   36.654 ms  29.490 ms  29.785 ms
 2   *192.168.1.2   60.319 ms (ICMP type:3, code:3, Destination port unreachable)

PC2> trace 192.168.1.3
trace to 192.168.1.3, 8 hops max, press Ctrl+C to stop
 1   192.168.0.1   25.218 ms  30.084 ms  29.478 ms
 2   *192.168.1.3   60.880 ms (ICMP type:3, code:3, Destination port unreachable)

PC2>
```

(b) `trace` hacia PC3 y PC4.

Figura 6: Pruebas de conectividad desde PC2 (1.d).

El `trace` de la Figura 6b confirma la ruta, con 192.168.0.1 como primer salto y el destino como segundo. VPCS antepone un asterisco a las respuestas que son mensajes ICMP de error, y el paréntesis indica cuál: tipo 3, código 3, puerto inalcanzable. Como VPCS sondea con UDP hacia un puerto cerrado, ese mensaje lo envía el propio PC3 o PC4 y prueba que la sonda llegó. Una sonda sin respuesta aparecería como un asterisco aislado, y aquí no hubo ninguna.

Los tiempos entre LANs, de 36,6 a 99,9 ms, los agrega el router emulado. El `trace` lo confirma, porque R1 tarda entre 25 y 37 ms en responder como primer salto, sin que el paquete llegue a la otra LAN. Dynamips emula por software la CPU MIPS del 3725 y, con el idle-PC configurado, la suspende cuando IOS está ocioso; ambas cosas retrasan el reenvío, aunque no se hicieron pruebas para separar cuánto aporta cada una.

En ninguno de los pings se perdió el primer paquete. Si R1 hubiera tenido que resolver por ARP la MAC del destino en ese momento, IOS habría descartado el paquete que disparó la consulta y VPCS habría mostrado `icmp_seq=1 timeout`. Lo más probable es que R1 ya conociera las MAC de PC3 y PC4 por el tráfico anterior, aunque su tabla ARP no se revisó antes de las pruebas.

1.2. Simulación de enlaces ideales y reales

La red de la Figura 3 del enunciado se armó en un proyecto GNS3 separado del de la pregunta 1. Los tres terminales son máquinas Alpine, creadas con el template que indica el material de ayuda (512 MB de RAM y `-cpu qemu64`), y el enlace con limitaciones es una VM NETem 0.4.1 intercalada entre Switch2 y PC3. PC1 está conectado a Switch1, PC2 a Switch2, y ambos switches están unidos entre sí. Las direcciones, de 192.168.0.1 a 192.168.0.3, se asignaron a mano con `ip addr add 192.168.0.x/24 dev eth0` e `ip link set eth0 up`. Además, a cada máquina se le cambió el nombre con `hostname` para distinguir las consolas en las capturas. En GNS3 los nodos conservan el nombre que les dio el template, de Alpine-ELO323-1 a Alpine-ELO323-3, y los cuadros de texto de la Figura 7 indican a qué PC corresponde cada uno y qué dirección tiene.

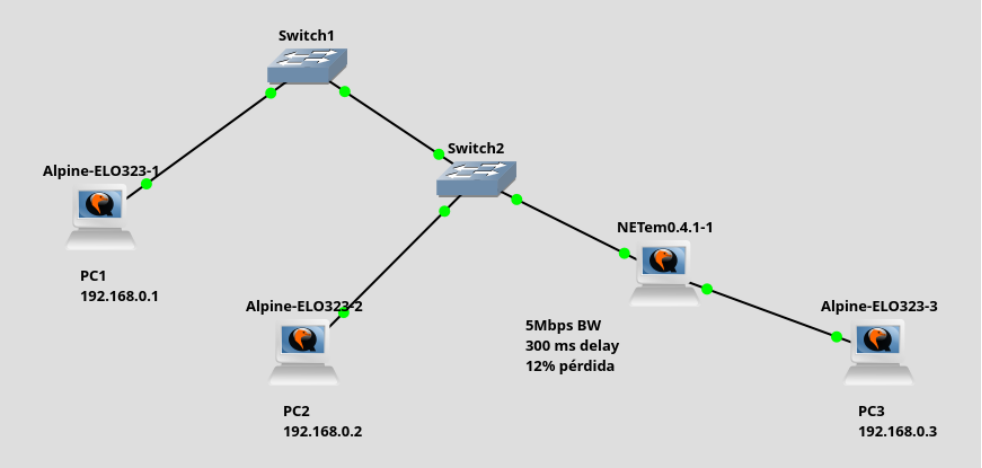


Figura 7: Topología de la pregunta 2 en GNS3, con las direcciones y la configuración del enlace NETem.

En el menú de NETem se configuró el sentido eth0 a eth1 con los valores de la Tabla 1 del enunciado (5000 kbit/s, 300 ms de retardo y 12 % de pérdida, sin jitter), y el sentido contrario quedó en la opción *Symmetric*, que copia la misma configuración. El Código 1 muestra cómo quedó aplicada en la VM. Cada interfaz tiene un limitador TBF a 5000 kbit/s y, debajo, un qdisc netem que agrega el retardo y la pérdida. La pérdida aparece como modelo de Gilbert-Elliott con $p = 12$ % y $r = 88$ %; como $p+r = 1$, la probabilidad de pasar al estado de pérdida es 12 % tanto desde el estado bueno como desde el malo, así que cada paquete se pierde con esa probabilidad sin importar lo que le pasó al anterior.

```
gns3@box:~$ tc qdisc show
qdisc tbf 1: dev eth0 root refcnt 2 rate 5000Kbit burst 1600b lat 20.0ms
qdisc netem 10: dev eth0 parent 1:1 limit 1000 delay 300.0msloss gemodel p 12% r 88% 1-h 0% 1-k 0%
qdisc tbf 1: dev eth1 root refcnt 2 rate 5000Kbit burst 1600b lat 20.0ms
qdisc netem 10: dev eth1 parent 1:1 limit 1000 delay 300.0msloss gemodel p 12% r 88% 1-h 0% 1-k 0%
```

Código 1: Configuración aplicada por NETem en sus dos interfaces.

1.2.1. Pruebas de conectividad (2.a)

Desde PC2, los diez pings a PC1 volvieron todos, con TTL 64 y tiempos entre 1,2 y 1,9 ms (Figura 8a). Hacia PC3 volvieron nueve de diez, y salvo el primero tardaron entre 602,3 y 603,3 ms. El retardo configurado se cumple: cada ping cruza NETem a la ida y a la vuelta, lo que suma $2 \times 300 = 600$ ms. Los 2 a 3 ms restantes corresponden al paso por los switches y las máquinas virtuales, que hacia PC1 ya toma entre 1,2 y 1,9 ms, más la transmisión de cada trama a 5 Mbit/s.

El primer ping a PC3 tardó 1205 ms porque PC2 todavía no conocía la MAC de PC3 y tuvo que pedirla por ARP. La consulta y su respuesta también cruzan NETem, así que el echo salió recién unos 600 ms después. El paquete perdido de los diez no sirve como estimación de la pérdida del enlace, porque con tan pocas muestras el resultado varía mucho de una prueba a otra; esa estimación se hace en la pregunta 2.b.

```
PC2:~# ping -c 10 192.168.0.1
PING 192.168.0.1 (192.168.0.1): 56 data bytes
64 bytes from 192.168.0.1: seq=0 ttl=64 time=1.359 ms
64 bytes from 192.168.0.1: seq=1 ttl=64 time=1.155 ms
64 bytes from 192.168.0.1: seq=2 ttl=64 time=1.659 ms
64 bytes from 192.168.0.1: seq=3 ttl=64 time=1.357 ms
64 bytes from 192.168.0.1: seq=4 ttl=64 time=1.268 ms
64 bytes from 192.168.0.1: seq=5 ttl=64 time=1.868 ms
64 bytes from 192.168.0.1: seq=6 ttl=64 time=1.319 ms
64 bytes from 192.168.0.1: seq=7 ttl=64 time=1.722 ms
64 bytes from 192.168.0.1: seq=8 ttl=64 time=1.410 ms
64 bytes from 192.168.0.1: seq=9 ttl=64 time=1.475 ms

--- 192.168.0.1 ping statistics ---
10 packets transmitted, 10 packets received, 0% packet loss
round-trip min/avg/max = 1.155/1.459/1.868 ms
PC2:~# ping -c 10 192.168.0.3
PING 192.168.0.3 (192.168.0.3): 56 data bytes
64 bytes from 192.168.0.3: seq=0 ttl=64 time=1205.236 ms
64 bytes from 192.168.0.3: seq=1 ttl=64 time=603.107 ms
64 bytes from 192.168.0.3: seq=2 ttl=64 time=602.997 ms
64 bytes from 192.168.0.3: seq=3 ttl=64 time=603.027 ms
64 bytes from 192.168.0.3: seq=5 ttl=64 time=602.771 ms
64 bytes from 192.168.0.3: seq=6 ttl=64 time=603.281 ms
64 bytes from 192.168.0.3: seq=7 ttl=64 time=602.809 ms
64 bytes from 192.168.0.3: seq=8 ttl=64 time=602.421 ms
64 bytes from 192.168.0.3: seq=9 ttl=64 time=602.318 ms

--- 192.168.0.3 ping statistics ---
10 packets transmitted, 9 packets received, 10% packet loss
round-trip min/avg/max = 602.318/669.774/1205.236 ms
PC2:~#
```

(a) Pings de PC2 a PC1 y PC3 (2.a).

```
PC2:~# ping -c 8000 -i 0.05 192.168.0.3 > p23.txt
PC2:~# tail -3 p23.txt
--- 192.168.0.3 ping statistics ---
8000 packets transmitted, 6165 packets received, 22% packet loss
round-trip min/avg/max = 600.607/601.345/614.882 ms
PC2:~# sh ic.sh p23.txt 8000
     N  perd    M(%)       s      t    t*s/vN    0.05M cumple
  1000   239   23.90  0.4267  1.962     2.65%    1.19% no
  2000   467   23.35  0.4232  1.961     1.86%    1.17% no
  3000   691   23.03  0.4211  1.961     1.51%    1.15% no
  4000   915   22.88  0.4201  1.961     1.30%    1.14% no
  5000  1156   23.12  0.4216  1.960     1.17%    1.16% no
  6000  1365   22.75  0.4193  1.960     1.06%    1.14% si
  7000  1598   22.83  0.4198  1.960     0.98%    1.14% si
  8000  1835   22.94  0.4205  1.960     0.92%    1.15% si
Primer N que cumple t*s/sqrt(N) <= 0.05*M: 5129
Perdida round-trip: M = 22.94%   IC95% = [22.02%, 23.86%]
Perdida por sentido: p = 1-sqrt(1-M) = 12.21%   IC95% = [11.69%, 12.74%]
PC2:~#
```

(b) Medición de pérdidas hacia PC3 y cálculo del intervalo (2.b).

Figura 8: Pruebas desde PC2 en la red con el enlace NETem.

1.2.2. Intervalos de confianza (2.b)

La estimación sigue el apunte del curso sobre pérdidas round-trip. Cada ping es un experimento binario, con $x_i = 1$ si el paquete se pierde, de modo que con k pérdidas en N envíos

$$M = \frac{k}{N}, \qquad s^2 = \frac{k}{N-1}\left(1 - \frac{k}{N}\right), \tag{1}$$

y el intervalo de confianza es $M \pm t\, s/\sqrt{N}$, con t de la distribución t de Student para $N-1$ grados de libertad y 95 % de confianza. El apunte define el intervalo de ± 5 % en forma relativa a la media, así que hay que reunir mediciones hasta que

$$t\,\frac{s}{\sqrt{N}} \le 0{,}05\, M. \tag{2}$$

Si NETem pierde 12 % en cada sentido, el ping debería perder $1-(1-0{,}12)^2 = 22{,}56$ % de los paquetes. Con ese valor y $t \approx 1{,}96$, la condición (2) se cumple desde unos 5300 envíos, de modo que se mandaron 8000 con `ping -c 8000 -i 0.05`. A 20 paquetes por segundo la prueba tomó unos siete minutos y generó unos 16 kbit/s, muy por debajo de los 5 Mbit/s del enlace, por lo que el limitador no descartó paquetes por su cuenta. La salida quedó en un archivo que procesa el script del Código 2: marca como perdido cada número de secuencia sin respuesta y cada 1000 envíos calcula M, s, t y el semiancho del intervalo. El valor de t sale de una expansión de la t de Student en función de los grados de libertad.

```
#!/bin/sh
# ic.sh <salida_de_ping> <N>: IC 95% de la perdida round-trip segun el apunte (experimento binario)
grep -o 'seq=[0-9]*' "$1" | cut -d= -f2 | awk -v N="$2" '
function tq(df,  z) { z = 1.959964; return z + (z*z*z + z)/(4*df) + (5*z*z*z*z*z + 16*z*z*z + 3*z)
    /(96*df*df) }
{ ok[$1] = 1 }
END {
  printf "%6s %5s %7s %7s %6s %9s %8s %s\n", "N", "perd", "M(%)", "s", "t", "t*s/vN", "0.05M", "
    cumple"
  k = 0; first = 0
  for (i = 0; i < N; i++) {
    if (!(i in ok)) k++
    n = i + 1
    if (n < 30) continue
```

```
    M = k/n; s = sqrt(k/(n-1)*(1 - k/n)); t = tq(n-1); h = t*s/sqrt(n)
    if (h <= 0.05*M && !first) first = n
    if (n % 1000 == 0) printf "%6d %5d %7.2f %7.4f %6.3f %8.2f%% %7.2f%% %s\n", n, k, 100*M, s, t,
    100*h, 5*M, (h <= 0.05*M ? "si" : "no")
  }
  printf "Primer N que cumple t*s/sqrt(N) <= 0.05*M: %d\n", first
  printf "Perdida round-trip: M = %.2f%%   IC95%% = [%.2f%%, %.2f%%]\n", 100*M, 100*(M-h), 100*(M+h)
  printf "Perdida por sentido: p = 1-sqrt(1-M) = %.2f%%   IC95%% = [%.2f%%, %.2f%%]\n", 100*(1-sqrt
    (1-M)), 100*(1-sqrt(1-(M-h))), 100*(1-sqrt(1-(M+h)))
}'
```

Código 2: Script `ic.sh` que calcula el intervalo de confianza en PC2.

Se perdieron 1835 de los 8000 paquetes (Figura 8b), lo que da $M = 22{,}94$ % con $s = 0{,}4205$ y $t = 1{,}960$. El semiancho del intervalo es de 0,92 puntos porcentuales, menor que $0{,}05\,M = 1{,}15$, y la condición se cumplió por primera vez con $N = 5129$; en la tabla, a los 5000 envíos todavía no se cumplía (1,17 frente a 1,16). Con 95 % de confianza, la pérdida de ida y vuelta está entre 22,02 % y 23,86 %.

El 22 % que informa ping es mayor que el 12 % configurado porque ping mide ida y vuelta y NETem aplica la pérdida en cada sentido. Un paquete que se pierde a la ida ya no puede perderse a la vuelta, de modo que la pérdida round-trip vale $P_{rt} = 2p - p^2$, algo menos que $2p$. Despejando, como indica el apunte,

$$p = 1 - \sqrt{1 - P_{rt}}, \tag{3}$$

se obtiene una pérdida por sentido de 12,21 %, con intervalo de 11,69 % a 12,74 % al aplicar la misma fórmula a los extremos. El 12 % configurado queda dentro, de modo que la pérdida medida en cada sentido es igual a la configurada con la precisión pedida. Lo mismo pasa con el valor esperado de ida y vuelta, 22,56 %, que cae dentro de su intervalo.

El intervalo supone que los envíos son independientes, y los datos lo confirman. La fracción de pérdidas fue 0,230 cuando el paquete anterior se había perdido y 0,229 cuando había llegado, y las rachas de 1, 2 y 3 pérdidas seguidas (1087, 252 y 59) coinciden con las que daría un proceso independiente con la misma tasa (1090, 250 y 57). Las 6165 respuestas tuvieron un RTT entre 600,6 y 614,9 ms, con promedio de 601,3 ms, lo que confirma también el retardo de 300 ms por sentido.

1.2.3. Mediciones de throughput (2.c)

En PC1 se dejó corriendo el servidor de iperf 2.2.0 con `iperf -s`, y desde PC2 y PC3 se ejecutó `iperf -c 192.168.0.1`, con `-t` para fijar la duración de la prueba e `-i` para que el cliente informe la tasa de cada intervalo. Por defecto iperf usa TCP: el cliente abre una conexión, escribe datos en ella tan rápido como TCP los acepta y, al terminar, cada extremo informa cuántos bytes pasaron y a qué tasa[1].

PC2 alcanzó 1,26 Gbit/s durante 30 s, con intervalos de 5 s entre 1,21 y 1,30 Gbit/s (Figura 9a). Su camino hacia PC1 solo cruza los dos switches, donde no hay ningún límite configurado. Cada trama viaja entre los procesos QEMU de las máquinas y los procesos dynamips que emulan los switches como un datagrama UDP del propio computador, así que esa tasa depende del procesamiento disponible y no de un enlace.

Desde PC3 se hicieron tres pruebas de 60 s (Tabla 2). Según el servidor, PC3 transfirió entre 465 y 686 KB, a 56,8, 62,6 y 74,1 kbit/s, con un promedio de 64,5 kbit/s. La tasa medida resultó mucho menor que los 5 Mbit/s configurados en NETem, apenas entre 1,1 y 1,5 % de ellos.

Tabla 2: Pruebas TCP de 60 s desde PC3 hacia PC1 (2.c).

	Cliente (PC3)		Servidor (PC1)		
Prueba	Duración (s)	Tasa (kbit/s)	Duración (s)	Recibido (KB)	Tasa (kbit/s)
1	77,91	72,1	75,85	686	74,1
2	67,72	56,3	67,10	465	56,8
3	73,68	62,1	73,06	559	62,6
Promedio					64,5

[1] Como en iperf, 1 KB son 1024 bytes y 1 kbit/s son 1000 bit/s.

En las tres pruebas el cliente siguió activo más de 60 s, hasta 77,9 s en la primera, y en la segunda y la tercera sus intervalos muestran bloques de 63,6 KB o cero bytes. Pasa porque el cliente cuenta los bytes que entrega a su socket TCP, no los que llegan a PC1. Mientras el búfer de envío tiene espacio la escritura es inmediata, y por eso en los primeros 10 s de la primera prueba el cliente alcanzó a escribir 368 KB (Figura 9b), aunque en ese lapso solo llegaron 225 KB a PC1. Cuando el búfer se llena, el cliente queda bloqueado hasta que TCP libera espacio. Al cumplirse los 60 s el búfer todavía guardaba datos y la conexión se cerró recién cuando terminaron de llegar. La tasa válida es entonces la que calcula el servidor (Figura 9c), que cuenta lo que efectivamente recibió.

```
PC2:~# clear; iperf -c 192.168.0.1 -t 30 -i 5
------------------------------------------------------------
Client connecting to 192.168.0.1, TCP port 5001
TCP window size: 45.0 KByte (default)
------------------------------------------------------------
[  1] local 192.168.0.2 port 59260 connected with 192.168.0.1 port 5001
[ ID] Interval       Transfer     Bandwidth
[  1] 0.00-5.00 sec   719 MBytes  1.21 Gbits/sec
[  1] 5.00-10.00 sec   730 MBytes  1.23 Gbits/sec
[  1] 10.00-15.00 sec   754 MBytes  1.27 Gbits/sec
[  1] 15.00-20.00 sec   751 MBytes  1.26 Gbits/sec
[  1] 20.00-25.00 sec   769 MBytes  1.29 Gbits/sec
[  1] 25.00-30.00 sec   777 MBytes  1.30 Gbits/sec
[  1] 0.00-30.02 sec  4.40 GBytes  1.26 Gbits/sec
PC2:~#
```

(a) Cliente en PC2.

```
PC3:~# iperf -c 192.168.0.1 -t 60 -i 10
------------------------------------------------------------
Client connecting to 192.168.0.1, TCP port 5001
TCP window size: 16.0 KByte (default)
------------------------------------------------------------
[  1] local 192.168.0.3 port 60160 connected with 192.168.0.1 port 5001
[ ID] Interval       Transfer     Bandwidth
[  1] 0.00-10.00 sec   368 KBytes   301 Kbits/sec
[  1] 10.00-20.00 sec  63.6 KBytes  52.1 Kbits/sec
[  1] 20.00-30.00 sec  63.6 KBytes  52.1 Kbits/sec
[  1] 30.00-40.00 sec  63.6 KBytes  52.1 Kbits/sec
[  1] 40.00-50.00 sec  63.6 KBytes  52.1 Kbits/sec
[  1] 50.00-60.00 sec  63.6 KBytes  52.1 Kbits/sec
[  1] 0.00-77.91 sec   686 KBytes  72.1 Kbits/sec
PC3:~#
```

(b) Cliente en PC3, primera prueba.

```
PC1:~# clear; iperf -s
------------------------------------------------------------
Server listening on TCP port 5001
TCP window size:  128 KByte (default)
------------------------------------------------------------
[  1] local 192.168.0.1 port 5001 connected with 192.168.0.2 port 59260
[ ID] Interval       Transfer     Bandwidth
[  1] 0.00-30.01 sec  4.40 GBytes  1.26 Gbits/sec
[  2] local 192.168.0.1 port 5001 connected with 192.168.0.3 port 60160
[ ID] Interval       Transfer     Bandwidth
[  2] 0.00-75.85 sec   686 KBytes  74.1 Kbits/sec
```

(c) Servidor en PC1.

Figura 9: Mediciones de throughput con TCP (2.c).

El límite de 5 Mbit/s no es lo que frena a TCP. Una conexión TCP envía como máximo una ventana de datos por cada RTT, así que su tasa es aproximadamente la ventana dividida por el RTT. Con 600 ms de RTT, llenar 5 Mbit/s exigiría tener $5 \cdot 10^6 \times 0{,}6/8 = 375\,000$ bytes en vuelo, unos 259 segmentos de 1448 bytes. Para ver qué ventana usó TCP en la práctica, durante la primera prueba se capturó el tráfico del enlace entre NETem y PC3 desde GNS3, y la captura se analizó con tshark.

PC3 envió 539 segmentos de datos, 486 distintos y 53 retransmisiones (9,8 % de los envíos, cerca del 12 % de pérdida configurado). TCP interpreta cada pérdida como una señal de congestión. Treinta y seis de ellas se recuperaron con retransmisión rápida, después de recibir ACK duplicados con bloques SACK, y en ese caso CUBIC, el control de congestión que usa Linux por defecto, reduce la ventana al 70 %. Las otras 17 se detectaron al vencer el temporizador de retransmisión (RTO), entre 0,6 y 2,5 s después del envío original, y entonces la ventana vuelve a un segmento. Como se pierde aproximadamente uno de cada ocho segmentos, la ventana no alcanza a crecer entre una reducción y la siguiente. En los primeros 10 s, mientras TCP crecía en *slow start* desde su ventana inicial de 10 segmentos, hubo en promedio 12,6 segmentos en vuelo (enviados y sin confirmar), con un máximo de 26. Desde ahí hasta el final el promedio bajó a 1,4 segmentos y nunca pasó de 10. La ventana que anunciaba PC1 estuvo siempre entre 43 y 318 KB, por encima de lo que había en vuelo, de modo que el receptor no fue el límite.

Los ACK de PC1 muestran el mismo cambio. Confirmaron 225 KB en los primeros 10 s, es decir 184 kbit/s, y entre 48 y 82 KB en cada intervalo de 10 s posterior, entre 39 y 67 kbit/s. Por eso la primera prueba promedió 74,1 kbit/s: el resultado de iperf incluye ese arranque, y en régimen permanente TCP se mantuvo cerca de 54 kbit/s. El modelo de Mathis et al.[2] estima la tasa de TCP en régimen con pérdidas

[2] M. Mathis, J. Semke, J. Mahdavi y T. Ott, "The macroscopic behavior of the TCP congestion avoidance algorithm", *ACM SIGCOMM Computer Communication Review*, vol. 27, n.º 3, pp. 67–82, 1997.

aleatorias como

$$B \approx \frac{\text{MSS}}{\text{RTT}} \cdot \frac{C}{\sqrt{p}}, \tag{4}$$

con $C = \sqrt{3/2}$ si el receptor confirma cada segmento y $C = \sqrt{3/4}$ si confirma uno de cada dos. Con MSS de 1448 bytes, RTT de 0,6 s y $p = 0{,}12$ se obtienen entre 48 y 68 kbit/s, y los 54 kbit/s medidos caen en ese rango. El modelo no considera los RTO, pero muestra lo esencial: en la expresión no aparece el ancho de banda del enlace, porque la tasa de TCP la determinan la pérdida y el RTT.

Para comprobar que el enlace sí entrega los 5 Mbit/s, la medición desde PC3 se repitió con UDP, que no tiene control de congestión y transmite a la tasa pedida sin esperar confirmaciones. En PC1 se levantó `iperf -s -u`, y PC3 envió datagramas de 1470 bytes durante 30 s, primero a 5 Mbit/s (`-u -b 5m`) y luego a 10 Mbit/s (Figura 10). El informe que el servidor devuelve al cliente llegó con la pérdida en 0 % en la primera prueba y no llegó en la segunda, así que los resultados se tomaron directamente del servidor.

```
PC3:~# iperf -c 192.168.0.1 -u -b 5m -t 30 -i 5
------------------------------------------------------------
Client connecting to 192.168.0.1, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size:  208 KByte (default)
------------------------------------------------------------
[  1] local 192.168.0.3 port 47038 connected with 192.168.0.1 port 5001
[ ID] Interval       Transfer     Bandwidth
[  1] 0.00-5.00 sec  2.98 MBytes  5.00 Mbits/sec
[  1] 5.00-10.00 sec  2.98 MBytes  5.00 Mbits/sec
[  1] 10.00-15.00 sec  2.98 MBytes  5.00 Mbits/sec
[  1] 15.00-20.00 sec  2.98 MBytes  5.00 Mbits/sec
[  1] 20.00-25.00 sec  2.98 MBytes  5.00 Mbits/sec
[  1] 25.00-30.00 sec  2.98 MBytes  5.00 Mbits/sec
[  1] 0.00-30.00 sec  17.9 MBytes  5.00 Mbits/sec
[  1] Sent 12759 datagrams
[  1] Server Report:
[ ID] Interval       Transfer     Bandwidth        Jitter   Lost/Total Datagrams
[  1] 0.00-30.05 sec  15.8 MBytes  4.41 Mbits/sec   0.000 ms 1486/12759 (0%)
PC3:~# iperf -c 192.168.0.1 -u -b 10m -t 30 -i 5
------------------------------------------------------------
Client connecting to 192.168.0.1, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size:  208 KByte (default)
------------------------------------------------------------
[  1] local 192.168.0.3 port 32796 connected with 192.168.0.1 port 5001
[ ID] Interval       Transfer     Bandwidth
[  1] 0.00-5.00 sec  5.96 MBytes  10.0 Mbits/sec
[  1] 5.00-10.00 sec  5.96 MBytes  10.0 Mbits/sec
[  1] 10.00-15.00 sec  5.96 MBytes  10.0 Mbits/sec
[  1] 15.00-20.00 sec  5.96 MBytes  10.0 Mbits/sec
[  1] 20.00-25.00 sec  5.96 MBytes  10.0 Mbits/sec
[  1] 25.00-30.00 sec  5.96 MBytes  10.0 Mbits/sec
[  1] 0.00-30.00 sec  35.8 MBytes  10.0 Mbits/sec
[  1] Sent 25515 datagrams
[  3] WARNING: did not receive ack of last datagram after 10 tries.
PC3:~#
```

(a) Cliente en PC3.

```
PC1:~# iperf -s -u
------------------------------------------------------------
Server listening on UDP port 5001
UDP buffer size:  208 KByte (default)
------------------------------------------------------------
[  1] local 192.168.0.1 port 5001 connected with 192.168.0.3 port 47038
[ ID] Interval       Transfer     Bandwidth        Jitter   Lost/Total Datagrams
[  1] 0.00-30.05 sec  15.8 MBytes  4.41 Mbits/sec   0.158 ms 1486/12759 (12%)
[  3] WARNING: ack of last datagram failed.
[  2] local 192.168.0.1 port 5001 connected with 192.168.0.3 port 32796
[ ID] Interval       Transfer     Bandwidth        Jitter   Lost/Total Datagrams
[  2] 0.00-32.12 sec  18.6 MBytes  4.86 Mbits/sec   0.631 ms 12241/25514 (48%)
[  4] WARNING: ack of last datagram failed.
```

(b) Servidor en PC1.

Figura 10: Mediciones de throughput con UDP desde PC3 (2.c).

A 5 Mbit/s llegaron 4,41 Mbit/s y se perdieron 1486 de 12759 datagramas, un 11,6 %, que coincide con el 12 % configurado (Figura 10b). No hubo otros descartes: NETem elimina cada datagrama con probabilidad 12 % antes de entregarlo al limitador, y lo que queda, unos 4,5 Mbit/s contando las cabeceras, no alcanza a llenar los 5000 kbit/s. A 10 Mbit/s llegaron 4,86 Mbit/s y se perdió el 48 % de los datagramas. El limitador TBF cuenta la trama completa, de 1512 bytes (1470 de datos, 8 de UDP, 20 de IP y 14 de Ethernet), y 5000 kbit/s de tramas equivalen a $5000 \times 1470/1512 = 4861$ kbit/s de datos, justo lo medido. Lo que excede esa tasa se acumula en la cola de NETem, de 1000 paquetes, y se descarta cuando la cola está llena. Esa misma cola explica que el servidor siguiera recibiendo unos 2 s después de que el cliente dejó de enviar.

Desde PC3, entonces, TCP obtuvo mucho menos que lo configurado, entre 57 y 74 kbit/s frente a 5 Mbit/s, aunque el enlace es capaz de entregarlo, como muestra el tope de 4,86 Mbit/s alcanzado con UDP. El valor de Bandwidth de NETem es un máximo, y TCP queda lejos de él porque, con 12 % de pérdida y 600 ms de RTT, su control de congestión mantiene la ventana en unos pocos segmentos. UDP, que no se frena, tampoco supera la tasa configurada: entrega 4,41 Mbit/s útiles al ofrecer 5 Mbit/s, por la pérdida, y nunca más de 4,86 Mbit/s, por las cabeceras.